Do Now · retrieval
Merge sort: trace and implement

Name: _____ Class: _____ Date: _____

Answer four retrieval questions before reviewing earlier learning.

Q1

In insertion sort, what happens to values greater than the key in the sorted prefix?

Q2

After one ascending bubble-sort pass on [4,1,3], what remains?

Q3

What is the length of int[] a={8,5,2,1}?

Q4

Insertion sort has sorted prefix [2,4,6,8] and next key 5. Trace the prefix after inserting the key, listing the values shifted right in order.

New Learning · key ideas

The important knowledge from today's lesson. Read, annotate, and explain it.

Core idea 1

Start with ordered runs — At each merge stage, both input runs must already be sorted.

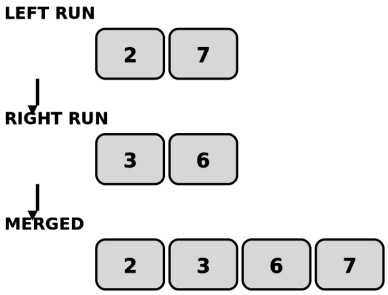
Core idea 2

Compare front values — Append the smaller current value, then advance that run's pointer.

Core idea 3

Copy the remainder — When a run finishes, copy the remainder. This code is the merge helper: top-down merge sort first recursively splits to length 0/1; bottom-up merge sort instead grows the run width.

Key visual · complete concept



Annotate the key visual

Explain which labels or features show the main process.

New Learning · applying the ideas

Vocabulary · use the correct technical terms

run — A consecutive sorted block.

merge — Combine sorted sequences while preserving order.

pointer — Position of the next candidate in a run.

bottom-up — Build sorted runs iteratively from small to large.

Check your understanding

Explain one key process in your own words using two of the terms above.

More animated examples are available in the original online lesson.

I Do · worked example

Problem

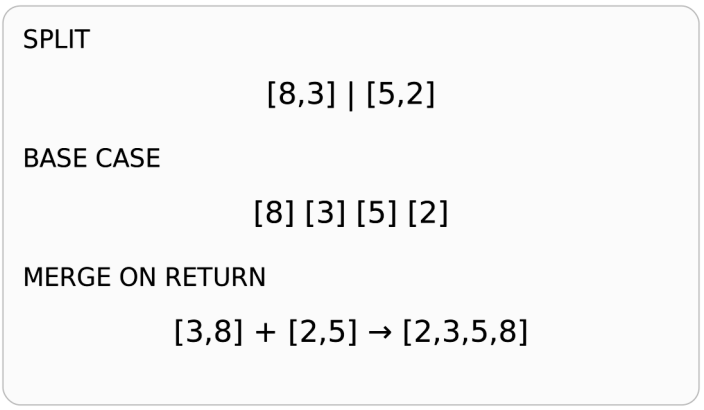
Trace recursive merge sort on [8,3,5,2]: split, reach the base case, then merge on return.

Follow the worked method

1. Split until each call is trivial — Split [8,3,5,2] into [8,3] and [5,2], then into [8],[3],[5],[2]. Each length-one call returns unchanged; an empty input also returns immediately.

2. Return through the merges — Merge [8] with [3]→[3,8]; [5] with [2]→[2,5]. Merge fronts 2,3,5, then copy8: [2,3,5,8]. Each helper input is already sorted. The outline uses end-exclusive copyRange and the merge loop above.

Source model · final illustrated step



Why this method works

Identify the decision, rule or invariant that makes the method correct.

We Do · Try → Discuss → Improve

TRY independently in BLACK PEN; compare methods with a partner; improve your reasoning.

Guided problem · TRY

Predict the call and return order
Pseudocode outline: MergeSort(a): if a.length <= 1: return a mid = a.length DIV 2 L = MergeSort(copyRange(a,0,mid)) R = MergeSort(copyRange(a,mid,a.length)) return MergeSorted(L,R)

DISCUSS

Compare how you reached the answer. Where could someone make an error?

IMPROVE

Add one precise correction or a stronger reason. Answers are not printed here.

You Do · independent practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 1

Merge [2,6] with [1,5].

Question 2

Recursively merge-sort [4,1,3,2]. Show both split levels, the base case and every merge on return.

Check that you have used the technical terms and explained your decisions.

You Do · extended practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 3

Explain the O(n log n) time complexity of merge sort.

Check that you have used the technical terms and explained your decisions.

Exit Ticket · reflect and retrieve

Exit Ticket

Two unsorted arrays can be passed to the standard merge step and always yield sorted output.

Use key vocabulary from memory

1. Explain run in your own words.

2. Explain merge in your own words.

3. Explain pointer in your own words.

Confidence and next step

How confident are you? Thumbs up / side / down

Which question would you improve after checking the classroom model?

Check answers after the lesson using the teacher-guided source models.